

Computer Organization and Architecture

500+ One-Liners for Rapid Revision

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Basic Computer Organization	5
Registers	5
Instruction Cycle	5
Machine Instructions	5
RISC vs CISC	6
Addressing Modes	6
Basic Organization - Additional	7
Functional Units and Buses	7
CPU Registers and Cycle Detail	7
Machine Instructions - Additional	7
Encoding and Formats	7
Addressing Modes - Additional	8
Effective Address and Use Cases	8
ALU and Data-path	8
ALU Design and Operations	8
Data-path and Micro-operations	8
Control Unit	9
Hardwired Control	9
Microprogrammed Control	9
Hardwired vs Microprogrammed Comparison	9
Computer Arithmetic	10
Number Representation	10
Overflow and Add/Subtract	10
Multiplication	10
Division	10
Floating Point (IEEE 754)	11
Pipelining	11
Concept and Stages	11
Performance Metrics	11
Hazards Overview	11

Structural Hazards	12
Data Hazards	12
Control / Branch Hazards	12
Advanced Pipelining	12
Memory Hierarchy	12
Hierarchy and Locality	12
Memory Types	13
Cache Memory	13
Cache Concept and Performance	13
Mapping Techniques	13
Field-Size Computation (Numericals)	14
Replacement Policies	14
Write Policies	14
Multi-level Cache	14
Main Memory and Secondary Storage	15
Memory Organization and Interleaving	15
Secondary Storage	15
I/O Organization	15
I/O Techniques Overview	15
Programmed I/O	16
Interrupt-Driven I/O	16
Direct Memory Access (DMA)	16
I/O Buses and Transfer	16
Computer Arithmetic (Extended)	16
Number Representation	17
Overflow and Carry	17
Multiplication and Division	17
Floating-Point (IEEE 754)	17
Instruction Set & Performance (Extended)	18
Performance Metrics	18
RISC vs CISC	18
Instruction Formats	18
Addressing Modes (Extended)	18
Pipelining (Extended)	18

Pipeline Performance	18
Pipeline Hazards	19
Memory Hierarchy (Extended)	19
Cache Mapping Numericals	19
Cache Policies	19
Main Memory	20
Control Unit (Extended)	20
Exam Rapid-Fire Numericals	20
System Buses & Data Paths (Extended)	20
Secondary Storage (Extended)	21
Parallelism & Misc (Extended)	21

Basic Computer Organization

- There are **5** functional units in a computer: **input**, **output**, **memory**, **ALU**, and **control unit**.
- The CPU has **3** core parts: the **register set**, the **ALU**, and the **control unit**.
- The **Von Neumann** architecture uses **1** shared memory and **1** bus for both instructions and data.
- The **Harvard** architecture uses **2** separate memories and **2** buses, one each for instructions and data.
- The **Von Neumann bottleneck** arises from the **single** shared bus limiting simultaneous instruction and data fetch.
- **Modified Harvard** architecture is used in modern CPUs with a unified memory but **split** L1 caches (I-cache and D-cache).
- The **stored-program concept** (Von Neumann) keeps **instructions and data** in the same read-write memory.
- The **system bus** has **3** types: **data bus**, **address bus**, and **control bus**.
- The **data bus** is **bidirectional** and its width equals the **word size**.
- The **address bus** is **unidirectional** and n lines address **2^n** locations.
- The **control bus** carries signals such as **Read, Write, Clock, Interrupt, and Bus grant**.
- **Bus arbitration** uses **3** methods: **daisy chaining**, **polling**, and **independent request**.
- The **ALU** performs **2** operation classes: **arithmetic** (add, subtract) and **logic** (AND, OR, NOT, XOR, shift).
- The **control unit** generates **control signals** that sequence all CPU operations.

Registers

- The **PC (Program Counter)** holds the **address of the next instruction** to be fetched.
- The **IR (Instruction Register)** holds the **currently executing instruction**.
- The **MAR (Memory Address Register)** holds the **memory address** to be accessed and its width equals the **address lines**.
- The **MDR/MBR** holds the **data** transferred to/from memory and its width equals the **word size**.
- The **accumulator** holds **intermediate ALU results** in accumulator-based machines.
- The **status/flag register (PSW)** holds **5** common flags: **Zero, Carry, Sign, Overflow, and Parity**.
- **2^{MAR}** gives the addressable memory size, while **MDR width** gives the word size.
- **General-purpose registers** (R0..Rn) are the **fastest** CPU storage with sub-nanosecond access.

Instruction Cycle

- The instruction cycle has **4** phases: **fetch**, **decode**, **execute**, and **interrupt**.
- During **fetch**, **4** micro-operations occur: **MAR<-PC**, **MDR<-M[MAR]**, **PC<-PC+1**, and **IR<-MDR**.
- The **PC <- PC + 1** increment happens in the **fetch** phase, not execute.
- **Interrupts** are checked **after** the execute phase of the current instruction.
- During the **interrupt** phase the CPU saves **PC and PSW** and loads the **ISR address**.

Machine Instructions

- A machine instruction has **2** basic fields: an **opcode** and zero or more **operand/address** fields.
- An m-bit opcode encodes up to **2^m** distinct operations.
- Instruction length is either **fixed** (RISC, e.g., **32 bits**) or **variable** (CISC, e.g., x86 **1-15 bytes**).
- An **expanding opcode** trades **operand bits** for **opcode bits** within a fixed-length instruction.
- There are **4** instruction formats by address count: **three-address**, **two-address**, **one-address**, and **zero-address**.
- A **three-address** instruction **ADD A, B, C** means **$A = B + C$** and gives the **shortest program**.

- A **two-address** instruction ADD A, B means **$A = A + B$** .
- A **one-address** instruction ADD X means **$AC = AC + X$** using the implied **accumulator**.
- A **zero-address** instruction ADD operates on the **stack** top using **postfix (RPN)** notation.
- Fewer addresses give **shorter instructions** but **longer programs**.
- Instruction types number **5**: **data transfer**, **arithmetic**, **logical**, **control transfer**, and **I/O**.
- **Data-transfer** instructions (LOAD, STORE, MOVE) move data with **no ALU operation**.
- **Control-transfer** instructions (JMP, CALL, RET) modify the **PC**.

RISC vs CISC

- **RISC** uses **fixed-length** instructions, **load-store** memory access, **many** registers, and **hardwired** control.
- **CISC** uses **variable-length** instructions, **memory operands**, **fewer** registers, and **microprogrammed** control.
- **RISC** targets **CPI ~ 1** , while **CISC** has **CPI > 1** .
- Examples of **RISC** are **ARM, MIPS, RISC-V**; examples of **CISC** are **x86 and VAX**.
- **RISC** pipelines more easily because of its **fixed-length** regular format.
- There are **3** CPU organizations: **accumulator** (1-address), **stack** (0-address), and **general-register** (2/3-address).

Addressing Modes

- An **addressing mode** defines how the **effective address (EA)** of an operand is computed.
- **Immediate** mode puts the operand **inside the instruction** and needs **0** memory references.
- **Direct (absolute)** mode sets **$EA = A$** and needs **1** memory reference.
- **Indirect** mode sets **$EA = M[A]$** and needs **2** memory references (the slowest).
- **Register** mode keeps the operand **in a register** with **0** memory references.
- **Register indirect** mode sets **$EA = (R)$** with **1** memory reference.
- **Displacement/based** mode sets **$EA = (R) + A$** .
- **Relative (PC-relative)** mode sets **$EA = (PC) + A$** and produces **position-independent** code.
- **Indexed** mode sets **$EA = A + (Xr)$** and is ideal for **array traversal**.
- **Auto-increment** mode uses **$EA = (R)$** then increments R by the **operand size**.
- **Auto-decrement** mode **decrements R first** then uses it as EA, natural for **stacks**.
- **Immediate** and **register** modes need **0** operand memory references; **indirect** needs **2**.
- The auto-increment step equals the **operand width** in bytes (**1, 2, 4, or 8**).
- **Relative, indexed, and based** modes all use **addition** but differ in what is added (PC, index, base).
- More addressing modes require **more mode bits**, shrinking the direct address range.

Basic Organization - Additional

Functional Units and Buses

- The **address bus width** determines the **maximum addressable memory**, so **32** address lines address **4 GB** (2^{32} bytes).
- A **16-bit** address bus addresses **64 KB** ($2^{16} = 65,536$ locations).
- A **20-bit** address bus addresses **1 MB** (2^{20}), as used by the original **8086**.
- The **word size** is the number of bits the CPU processes in **1** operation and equals the **data bus / register width**.
- Bus bandwidth equals **bus width x bus frequency** and is measured in **bytes/second**.
- **Daisy chaining** arbitration grants the bus by **priority based on physical position**, with the nearest device having **highest** priority.
- **Polling** arbitration uses a **count/poll line** and priority is set by the **polling order** (programmable).
- **Independent request** arbitration uses **separate request and grant lines** per device and is the **fastest** but needs the **most** control lines.
- A **synchronous bus** uses a **common clock**, while an **asynchronous bus** uses **handshaking** signals.
- The **3** bus types differ in direction: **address bus** is unidirectional, **data bus** is bidirectional, and **control bus** is mixed.

CPU Registers and Cycle Detail

- The **instruction cycle** time equals the **fetch cycle + execute cycle** and may include an **indirect** and **interrupt** cycle.
- The **indirect cycle** occurs between **decode and execute** when the instruction uses **indirect addressing**.
- A **memory reference** requires loading **MAR** then transferring via **MDR** over the **data bus**.
- **Register-to-register** operations are the **fastest** because they avoid **main memory** access.
- The **stack pointer (SP)** holds the **address of the top** of the stack and changes on **PUSH/POP**.
- A **full-descending** stack means SP points to the **last used** location and grows toward **lower** addresses.
- The **PSW (Program Status Word)** combines the **flag register** and **processor mode** bits.
- The **micro-operation** is the **smallest** CPU operation, executed in **1** clock pulse.
- **Register transfer notation (RTN)** uses the form **R2 <- R1** to denote a **data transfer**.
- The **4** register categories by use are **user-visible**, **control/status**, **address**, and **data** registers.

Machine Instructions - Additional

Encoding and Formats

- A **stack machine** (0-address) evaluates the expression **(A+B)*C** using postfix **AB+C***.
- Converting **X = (A+B)*(C+D)** on a 2-address machine needs at least **3** instructions using a temporary.
- An **8-bit opcode** field supports **256** distinct instructions; reducing operands enables **expanding** opcodes.
- The **opcode** specifies the **operation**, while the **mode field** specifies the **addressing mode**.
- **Variable-length** instructions improve **code density** but complicate **decoding and pipelining**.
- **Fixed-length** instructions (e.g., **ARM/MIPS 32-bit**) simplify **fetch and pipelining** but waste **some space**.
- A **memory-to-memory** architecture allows **both operands and result** in memory, increasing **memory traffic**.

- The **number of operands** explicitly named decreases from **3 to 0** as machines move from register to stack organization.
- An **instruction set architecture (ISA)** is the **programmer-visible** interface: instructions, registers, and addressing modes.
- **Orthogonality** in an ISA means **any addressing mode** can be used with **any opcode** independently.

Addressing Modes - Additional

Effective Address and Use Cases

- **Implied/implicit** mode names **no operand** explicitly; the operand is **built into the opcode** (e.g., **CLC**, **CMA**).
- **Immediate** mode is the **fastest** for constants because the operand needs **0** memory accesses.
- **Direct** mode limits the address range to the **address-field width**, e.g., a **12-bit** field reaches **4096** locations.
- **Indirect** mode enables **pointers** and needs **2** memory accesses, making it the **slowest**.
- **Register indirect** mode is like indirect but the pointer is in a **register**, needing only **1** memory access.
- **Indexed** mode computes **EA = base address + index register**, perfect for stepping through an **array**.
- **Base-register** mode computes **EA = base register + displacement** and supports **relocation** of programs.
- **PC-relative** mode ($EA = PC + \text{offset}$) is used by **branch** instructions to create **position-independent code**.
- **Auto-increment** mode advances the pointer by the **operand size**, ideal for **sequential array access**.
- **Auto-decrement** mode is the natural fit for implementing a **stack PUSH** in **full-descending** order.
- For an instruction at address **2000** with offset **50**, PC-relative EA (PC already incremented to **2001**) is **2051**.
- **Scaled indexed** mode computes **EA = base + index * scale**, where scale is the **element size** (1,2,4,8).
- The **2** modes with the fewest memory references (0) are **immediate** and **register**.
- **Relative addressing** produces **shorter** branch encodings because offsets are **small** signed values.
- Among all modes, **memory-indirect** requires the **most** (2) operand memory accesses.

ALU and Data-path

ALU Design and Operations

- The **ALU** performs **2** broad classes of operations: **arithmetic** and **logical**, selected by **control/function-select** lines.
- An n-bit ALU with **k** function-select lines can perform up to **2^k** operations.
- The **74181** is a classic **4-bit** ALU slice supporting **16 arithmetic** and **16 logic** functions.
- A **ripple-carry adder** of n bits has a worst-case delay proportional to **$O(n)$** due to **carry propagation**.
- A **carry-lookahead adder (CLA)** reduces delay to about **$O(\log n)$** using **generate (G)** and **propagate (P)** signals.
- Carry generate is **$G_i = A_i \cdot B_i$** and carry propagate is **$P_i = A_i \oplus B_i$** .
- The **ALU status flags** produced are typically **4: Zero, Carry, Sign/Negative, and Overflow**.
- A **barrel shifter** performs an n-bit shift/rotate in **1** clock cycle using **combinational logic**.
- **Logical shift** fills with **0**, while **arithmetic right shift** preserves the **sign bit**.
- The **shifter** is part of the ALU and supports **4** operations: **shift left, shift right, rotate left, rotate right**.

Data-path and Micro-operations

- A single-bus data-path allows only **1** register transfer per clock, increasing **cycle count**.
- A **three-bus** data-path lets an ALU operation (read 2 operands, write result) finish in **1** clock cycle.

- A **micro-operation** is an elementary operation executed on data in registers during **1** clock pulse.
- The **4** micro-operation categories are **register transfer**, **arithmetic**, **logic**, and **shift**.
- The fetch micro-operations conventionally take **3** time-steps (T0, T1, T2) in a single-bus CPU.
- A **bus** in the data-path is built from **tri-state buffers** or **multiplexers** to select the source register.
- The **ALU**, **register file**, and **interconnect buses** are the **3** core data-path components.
- A temporary register like **Y** or **Z** holds an ALU operand because the bus carries only **1** value at a time.
- **RTL (Register Transfer Language)** expresses control as conditional transfers like **T1: MAR <- PC**.
- Splitting fetch and execute increases **throughput** only when overlapped via **pipelining**.

Control Unit

Hardwired Control

- The **control unit** generates **control signals** in the correct **sequence and timing** to execute instructions.
- A **hardwired** control unit implements control logic with **fixed combinational circuits**, flip-flops, and a **sequence counter**.
- The **4** inputs to a hardwired control unit are the **IR (opcode)**, **timing signals**, **flags**, and **clock**.
- A hardwired control unit is **fast** but **difficult to modify**, making it ideal for **RISC** CPUs.
- A **sequence counter (SC)** with a **decoder** produces the **timing signals (T0..Tn)** in hardwired control.
- The **3** design methods for hardwired control are **state-table**, **delay-element/sequence-counter**, and **PLA-based**.
- Hardwired control changes to the instruction set require **redesigning the hardware**.

Microprogrammed Control

- A **microprogrammed** control unit stores control signals as **microinstructions** in a **control memory (control store)**, usually **ROM**.
- A **microinstruction** contains the **control-signal (microoperation) field** and the **next-address** field.
- A **microprogram** is the **sequence of microinstructions** that implements one machine instruction.
- The **control address register (CAR/CMAR)** holds the address of the **next microinstruction** in control memory.
- The **control buffer register (CBR/CMDBR)** holds the **current microinstruction** fetched from control memory.
- **Microprogrammed** control is **flexible and easy to modify** but **slower** than hardwired, suiting **CISC** CPUs.
- The **2** microprogramming styles are **horizontal** and **vertical** microcode.
- **Horizontal microcode** uses **one bit per control signal** (unencoded), giving **high parallelism** but **wide** words.
- **Vertical microcode encodes** control fields, giving **narrow** words but needing a **decoder** and less parallelism.
- **Nanoprogramming** uses **2** levels of control store to reduce **control memory size**.
- **Microprogram sequencing** determines the next address via **4** methods: **increment**, **branch**, **conditional branch**, and **mapping**.
- **Opcode mapping** converts the machine **opcode** into the **starting address** of its microroutine.
- The **control memory** size equals **number of microinstructions x microinstruction width**.
- A **horizontal** microinstruction of width 64 bits can directly drive up to **64** control points.

Hardwired vs Microprogrammed Comparison

- **Hardwired** control is **faster**; **microprogrammed** control is **slower** due to **control-memory access**.
- **Hardwired** uses **combinational logic**; **microprogrammed** uses **control memory (ROM)**.
- **Hardwired** is **hard to modify**; **microprogrammed** is **easy to modify** by changing microcode.

- **Hardwired** suits **RISC/simple** ISAs; **microprogrammed** suits **CISC/complex** ISAs.
- **Hardwired** needs **less** chip area for simple sets but **more design effort** for complex ones.
- The **decode-execute** logic in hardwired control is realized by a **decoder + control matrix**.
- **Microprogrammed** control simplifies **diagnostics and emulation** of other machines.
- Adding a new instruction in microprogrammed control means **adding a microroutine**, not rewiring.

Computer Arithmetic

Number Representation

- **Fixed-point** representation places the **binary point** at a **fixed** position, trading **range for simplicity**.
- The **3** signed-integer representations are **sign-magnitude**, **1's complement**, and **2's complement**.
- **Sign-magnitude** uses the **MSB** as the sign and has **2** representations of zero (**+0 and -0**).
- **1's complement** negates by **flipping all bits** and also has **2** zeros (**+0 and -0**).
- **2's complement** negates by **flipping all bits and adding 1** and has a **single** zero.
- An n-bit **2's complement** number ranges from $-2^{(n-1)}$ to $2^{(n-1)} - 1$.
- An **8-bit 2's complement** range is **-128 to +127**.
- An n-bit **unsigned** number ranges from **0** to $2^n - 1$.
- **2's complement** is preferred because it uses **1** adder for both **addition and subtraction**.
- To subtract $A - B$ in 2's complement, **add A and the 2's complement of B**.
- The **1's complement** end-around-carry requires **adding the carry-out back** to the LSB.
- **Sign extension** of a 2's complement number replicates the **sign bit** into the new high-order bits.

Overflow and Add/Subtract

- **Overflow** in 2's complement occurs when **two same-sign operands** produce an **opposite-sign result**.
- Overflow can be detected when the **carry into** the sign bit differs from the **carry out** of the sign bit (their **XOR** = 1).
- Adding two **positive** numbers giving a **negative** result indicates **overflow**.
- **Unsigned overflow** is indicated by the **carry-out** flag, while **signed overflow** uses the **overflow (V)** flag.
- For **n-bit** addition the result may need **n+1** bits, with the extra bit being the **carry-out**.

Multiplication

- **Booth's algorithm** multiplies **signed 2's complement** numbers by examining **2** bits (**Q_n, Q_{n+1}**) at a time.
- Booth's recoding rules: **10 -> subtract**, **01 -> add**, and **00 or 11 -> no operation** (just shift).
- Booth's algorithm is efficient for operands with **long runs** of **1s or 0s**.
- An $n \times n$ **array multiplier** uses n^2 AND gates and **(n-1)** rows of adders, with delay **$O(n)$** .
- **Sequential (shift-add) multiplication** of n-bit numbers takes about **n** add-shift cycles.
- The product of two n-bit numbers can require up to **2n** bits.
- **Wallace tree** multipliers reduce partial-sum addition delay to **$O(\log n)$** .

Division

- The **2** bit-at-a-time division methods are **restoring** and **non-restoring** division.
- **Restoring division** **restores** the partial remainder by **adding back** the divisor when the subtraction is negative.
- **Non-restoring division** avoids the restore step, doing **add or subtract** on the next step, and is **faster**.
- Dividing an **2n-bit** dividend by an n-bit divisor yields an n-bit **quotient** and n-bit **remainder**.

- **Division by zero** raises a **hardware exception/trap**.

Floating Point (IEEE 754)

- **IEEE 754** defines **3** fields: **sign**, **exponent (biased)**, and **mantissa/significand**.
- **Single precision** uses **32 bits**: **1** sign, **8** exponent, **23** mantissa bits.
- **Double precision** uses **64 bits**: **1** sign, **11** exponent, **52** mantissa bits.
- The single-precision exponent bias is **127**; the double-precision bias is **1023**.
- The **normalized** mantissa has an **implicit/hidden leading 1**, giving **24** effective bits in single precision.
- The value of a normalized IEEE number is $(-1)^S \times 1.M \times 2^{(E-bias)}$.
- An all-zero exponent with nonzero mantissa denotes a **denormalized/subnormal** number.
- An all-ones exponent with zero mantissa denotes **infinity**; with nonzero mantissa denotes **NaN**.
- Floating-point **addition** requires **4** steps: **align exponents**, **add mantissas**, **normalize**, and **round**.
- **Single precision** provides about **7** decimal digits of precision; double provides about **15-16**.
- **Guard, round, and sticky** bits (**3** extra bits) improve floating-point **rounding** accuracy.
- The **default** IEEE 754 rounding mode is **round to nearest, ties to even**.

Pipelining

Concept and Stages

- **Pipelining** overlaps instruction execution by dividing it into **stages** like an **assembly line**.
- The classic **RISC** pipeline has **5** stages: **IF, ID, EX, MEM, WB**.
- The **5** stages expand to **Instruction Fetch, Instruction Decode, Execute, Memory access, Write Back**.
- A k-stage pipeline can have up to **k** instructions **in flight** simultaneously.
- The **pipeline register/latch** between stages holds **intermediate results** and synchronizes to the **clock**.
- The **clock cycle time** of a pipeline equals the **slowest stage delay** plus the **latch delay**.
- **Balancing** stages so all have **equal delay** maximizes pipeline **throughput**.
- The **first** instruction in a k-stage pipeline completes after **k** cycles (fill latency).

Performance Metrics

- For n instructions in a k-stage pipeline, total cycles = **k + (n - 1)** (ideal, no stalls).
- The ideal pipeline **speedup** approaches **k** (number of stages) for **large n**.
- Exact speedup = $(n \times k) / (k + n - 1)$ compared to a non-pipelined k-cycle processor.
- **Throughput** of an ideal pipeline approaches **1 instruction per clock cycle (IPC = 1)**.
- Pipeline **efficiency** = **speedup / number of stages**, approaching **1** (100%) for large n.
- The maximum theoretical speedup of a k-stage pipeline is **k**.
- **CPI (cycles per instruction)** of an ideal pipeline is **1**; stalls raise it above 1.
- Effective CPI = **1 + stall cycles per instruction**.
- A non-pipelined processor needs **n x k** cycles for n k-stage instructions.
- For 100 instructions on a 5-stage pipeline (ideal), total cycles = **104** (5 + 99).
- The **speedup** from pipelining is limited by **hazards** and **stage imbalance**.

Hazards Overview

- Pipeline **hazards** come in **3** types: **structural**, **data**, and **control (branch)**.
- A **hazard** forces the pipeline to **stall (insert bubbles)**, raising CPI above the ideal **1**.

Structural Hazards

- A **structural hazard** occurs when **2** instructions need the **same hardware resource** in the same cycle.
- A common structural hazard is a **single memory port** serving both **IF** and **MEM** simultaneously.
- **Structural hazards** are removed by **duplicating resources** or using **separate instruction/data caches (Harvard)**.

Data Hazards

- The **3** data hazard types are **RAW (read after write)**, **WAR (write after read)**, and **WAW (write after write)**.
- **RAW** is a **true dependency** and the **most common** hazard in in-order pipelines.
- **WAR (anti-dependency)** and **WAW (output dependency)** appear mainly with **out-of-order** execution.
- **Forwarding/bypassing** resolves many RAW hazards by routing the ALU result **directly to the next stage**.
- A **load-use hazard** needs at least **1** stall cycle even with forwarding because data is ready only after **MEM**.
- **Stalling (bubbles)** resolves data hazards by **delaying** dependent instructions.
- **Operand forwarding** reduces stalls but cannot eliminate the **load-delay** slot.
- Software can hide hazards via **instruction scheduling/reordering** by the **compiler**.

Control / Branch Hazards

- A **control (branch) hazard** arises because the **branch outcome** is unknown during **IF** of following instructions.
- A taken branch in a 5-stage pipeline can cause a **branch penalty** of up to **3** flushed instructions.
- **Branch prediction** reduces control hazards via **static** or **dynamic** schemes.
- **Static prediction** examples: **always taken**, **always not-taken**, and **backward-taken/forward-not-taken (BTFN)**.
- **Dynamic prediction** uses a **branch history table (BHT)** with **1-bit or 2-bit** saturating counters.
- A **2-bit** predictor must mispredict **twice** before changing its prediction direction.
- The **branch target buffer (BTB)** caches **predicted target addresses** to avoid stalls.
- A **delayed branch** fills the **branch delay slot** with a useful instruction (used in **MIPS**).
- **Branch penalty cycles** = mispredict cycles x branch frequency x mispredict rate.
- **Flushing/squashing** discards **wrongly fetched** instructions after a mispredicted branch.

Advanced Pipelining

- A **superscalar** processor issues **multiple instructions per cycle**, achieving $CPI < 1$ ($IPC > 1$).
- **Out-of-order execution** uses **Tomasulo's algorithm** with **reservation stations** and **register renaming**.
- **Register renaming** eliminates **WAR and WAW** (false) dependencies.
- **Superpipelining** uses **deeper** pipelines with **shorter** stages and a **higher** clock.
- A **VLIW** processor relies on the **compiler** to pack **independent operations** into one long instruction word.
- **In-order issue, out-of-order completion** can create **WAW** hazards.
- The **reorder buffer (ROB)** restores **in-order** commit for **precise exceptions**.
- **Speculative execution** executes instructions **before** a branch is resolved, discarding results if **mispredicted**.

Memory Hierarchy

Hierarchy and Locality

- The memory hierarchy has **5** levels from fastest to slowest: **registers, cache, main memory, secondary, tertiary**.
- Going down the hierarchy, **speed and cost/bit decrease** while **capacity increases**.
- **Locality of reference** has **2** forms: **temporal locality** and **spatial locality**.
- **Temporal locality** means a recently accessed item is **likely accessed again soon** (e.g., **loop variables**).
- **Spatial locality** means items near a referenced address are **likely accessed soon** (e.g., **array elements**).
- **Cache** exploits **temporal** locality via retention and **spatial** locality via **block/line** transfer.
- The **memory hierarchy** aims to give the **speed of cache** at the **cost of main memory**.

Memory Types

- **RAM** is **volatile** read/write memory with **2** types: **SRAM** and **DRAM**.
- **SRAM** stores bits in **flip-flops (6 transistors)**, is **fast**, and needs **no refresh** (used for **cache**).
- **DRAM** stores bits in **1 transistor + 1 capacitor**, is **dense/cheap**, and needs **periodic refresh** (used for **main memory**).
- **DRAM refresh** is required because the **capacitor charge leaks** over time (typically every **~64 ms**).
- **ROM** is **non-volatile** with **4** variants: **Mask ROM, PROM, EPROM, EEPROM**.
- **PROM** is programmed **once** by the user; **EPROM** is erased by **UV light**; **EEPROM** is erased **electrically**.
- **Flash memory** is a type of **EEPROM** erased in **blocks**, used in **SSDs and USB drives**.
- **SRAM** is **faster but costlier and less dense** than **DRAM**.
- **SDRAM** synchronizes with the **system clock**; **DDR** transfers data on **both clock edges**.

Cache Memory

Cache Concept and Performance

- A **cache hit** means the requested data is **found in cache**; a **miss** means it must be **fetches from a lower level**.
- **Hit ratio (H) = number of hits / total accesses**; **miss ratio = 1 - H**.
- **AMAT (average memory access time) = hit time + miss rate x miss penalty**.
- For hit time **1 ns**, miss rate **0.05**, miss penalty **100 ns**: $AMAT = 1 + 0.05 \times 100 = 6 \text{ ns}$.
- **Effective access time = H x cache time + (1 - H) x (cache time + memory time)** for a serial lookup.
- For **H = 0.9**, cache = **10 ns**, memory = **100 ns** (serial): $EAT = 10 + 0.1 \times 100 = 20 \text{ ns}$.
- **Multilevel AMAT = HitTimeL1 + MissRateL1 x (HitTimeL2 + MissRateL2 x MissPenaltyL2)**.
- The **3 C's** of cache misses are **compulsory (cold)**, **capacity**, and **conflict** misses.
- **Compulsory misses** happen on the **first reference** to a block and are reduced by **larger block size/prefetching**.
- **Conflict misses** occur in **direct-mapped/set-associative** caches and are reduced by **higher associativity**.
- **Capacity misses** occur because the cache **cannot hold** the working set and are reduced by a **larger cache**.

Mapping Techniques

- The **3** cache mapping techniques are **direct mapping, fully associative, and set associative**.
- In **direct mapping**, block **j** maps to **exactly 1** cache line = **(block number) mod (number of lines)**.
- **Direct-mapped** address splits into **3** fields: **Tag, Line/Index, and Block offset (word)**.
- **Direct mapping** is **simplest and cheapest** but has the **most conflict misses**.
- **Fully associative** mapping lets a block go into **any** cache line, needing only **Tag + offset** fields.
- **Fully associative** has the **fewest conflict misses** but needs **costly parallel tag comparison (CAM)**.

- **Set associative** mapping groups lines into **sets**; a block maps to **1 set** but **any line within it**.
- **Set-associative** address splits into **3** fields: **Tag**, **Set/Index**, and **Block offset**.
- An **n-way** set-associative cache has **n** lines per set and needs **n** parallel comparators.
- **Direct mapping** is a **1-way** set-associative cache; **fully associative** is an **m-way** (all lines in 1 set).
- The set number in set-associative mapping = **(block number) mod (number of sets)**.

Field-Size Computation (Numericals)

- **Block offset bits** = $\log_2(\text{block size in words/bytes})$.
- **Line/index bits (direct)** = $\log_2(\text{number of cache lines})$.
- **Set index bits (set-associative)** = $\log_2(\text{number of sets})$.
- **Tag bits** = **(total address bits) - (index bits) - (offset bits)**.
- **Number of cache lines** = **cache size / block size**.
- **Number of sets** = **number of lines / associativity**.
- For a **32-bit** address, **16 KB** direct cache, **16-byte** blocks: offset = **4**, index = **10**, tag = **18** bits.
- That cache has **1024** lines (16 KB / 16 B) so **10** index bits.
- For a **4-way**, **16 KB**, **16-byte**-block cache: lines = **1024**, sets = **256**, set bits = **8**.
- **Doubling associativity** (halving sets) moves **1** bit from **index** to **tag**.
- **Doubling block size** moves **1** bit from **index** to **offset** (cache size fixed).
- **Fully associative** mapping uses **0** index bits; the entire non-offset address is the **tag**.
- A **valid bit** marks whether a cache line holds **valid data**; a **dirty bit** marks **modified** data.

Replacement Policies

- The **4** cache replacement policies are **LRU**, **FIFO**, **Random**, and **Optimal (OPT/Belady's)**.
- **LRU (Least Recently Used)** evicts the block **unused for the longest** time, exploiting **temporal locality**.
- **FIFO** evicts the **oldest loaded** block and can suffer **Belady's anomaly**.
- **Optimal (Belady's)** evicts the block used **furthest in the future** and is a **theoretical** lower bound.
- **Random** replacement needs the **least hardware** and avoids **worst-case patterns**.
- **Direct-mapped** caches need **no replacement policy** because each block has **only 1** possible line.
- **LRU** for an n-way set needs about $\log_2(n!)$ bits or $n \times n$ bits per set to track order.

Write Policies

- The **2** write policies are **write-through** and **write-back**.
- **Write-through** updates **both cache and memory** on every write, keeping them **consistent** but causing more **memory traffic**.
- **Write-back** updates **only the cache** and writes to memory on **eviction**, using a **dirty bit**.
- **Write-back** reduces **memory traffic** but risks **inconsistency** on crashes.
- On a write miss, **write-allocate** loads the block into cache; **no-write-allocate** writes **directly to memory**.
- **Write-through** is usually paired with **no-write-allocate**; **write-back** with **write-allocate**.
- A **write buffer** lets the CPU continue while writes drain to memory, hiding **write-through** latency.

Multi-level Cache

- Modern CPUs use **3** cache levels: **L1** (smallest/fastest), **L2**, and **L3** (largest/shared).
- **L1** cache is often **split** into **I-cache** and **D-cache** (Harvard), while **L2/L3** are usually **unified**.
- **Local miss rate** = misses in that level / accesses to that level; **global miss rate** = misses / total CPU accesses.
- A **multi-level** hierarchy lowers **effective miss penalty** by catching misses in the **next level**.

- **Inclusive** caches duplicate L1 contents in L2; **exclusive** caches do not, saving **space**.

Main Memory and Secondary Storage

Memory Organization and Interleaving

- **Memory interleaving** spreads consecutive addresses across **multiple memory banks** to allow **parallel access**.
- The **2** interleaving schemes are **low-order (consecutive addresses in different banks)** and **high-order interleaving**.
- **Low-order interleaving** maximizes **bandwidth** for sequential access (good for **cache block fetch**).
- **High-order interleaving** keeps consecutive addresses in the **same bank**, aiding **fault isolation**.
- With **m** banks, low-order interleaving can deliver up to **m** words per access cycle.
- A memory of 2^n words x **m** bits needs **n** address lines and **m** data lines.
- To build a larger memory from chips, combine them in **2** ways: **horizontal (word-width)** and **vertical (address-space)** expansion.
- Building a **16K x 8** memory from **2K x 8** chips needs **8** chips and a **3-to-8 decoder** for chip select.
- A **1K x 4** chip provides **1024** addresses and **4** data bits each.
- **Address decoding** uses the **high-order** address bits to select a **chip/bank**.

Secondary Storage

- **HDD** access time has **3** components: **seek time**, **rotational latency**, and **transfer time**.
- **Seek time** is the time to move the **head** to the correct **track/cylinder**.
- **Rotational latency** averages **half** a disk rotation; at **7200 RPM** it is about **4.17 ms**.
- **Average rotational latency** = $(1/2) \times (60 / \text{RPM})$ seconds.
- **Disk capacity** = **heads x tracks/surface x sectors/track x bytes/sector**.
- **SSD** uses **NAND flash**, has **no moving parts**, and offers **lower latency** than HDD.
- **SSDs** have **limited write endurance** (P/E cycles) and use **wear leveling**.
- A disk's **data transfer rate** = **(bytes per track) x (rotations per second)**.
- **HDD** is **cheaper per GB** but **slower**; **SSD** is **faster** but **costlier per GB**.
- **Virtual memory** uses secondary storage (swap/page file) to extend **main memory**, linked to the **OS**.

I/O Organization

I/O Techniques Overview

- The **3** I/O data-transfer methods are **programmed I/O**, **interrupt-driven I/O**, and **DMA**.
- **Programmed I/O** keeps the CPU **fully busy polling**, wasting **CPU cycles**.
- **Interrupt-driven I/O** frees the CPU between transfers but still moves data **1 word per interrupt** via the CPU.
- **DMA** transfers data **directly between device and memory**, bypassing the CPU for the **bulk transfer**.
- **DMA** is best for **large/block** transfers; **interrupt-driven** suits **moderate** rates; **programmed** suits **simple/slow** devices.
- The **2** I/O addressing schemes are **memory-mapped I/O** and **isolated (I/O-mapped) I/O**.
- **Memory-mapped I/O** shares the **address space** with memory and uses **normal load/store** instructions.
- **Isolated I/O** uses a **separate I/O address space** with dedicated **IN/OUT** instructions and an **IO/M** control line.

Programmed I/O

- **Programmed I/O** repeatedly tests a **status flag/bit** in a loop called **polling**.
- An **I/O module** has **3** register types: **data**, **status**, and **control/command** registers.
- Programmed I/O wastes CPU time because of **busy-waiting** on slow devices.
- In programmed I/O, the **CPU** is responsible for **every** word transfer.

Interrupt-Driven I/O

- An **interrupt** suspends the CPU, which saves **PC and PSW**, runs an **ISR**, then restores state.
- Interrupts are classified as **maskable** or **non-maskable (NMI)**.
- **Maskable** interrupts can be **disabled/ignored** via the **interrupt-enable flag**; **NMI** cannot be.
- Interrupts are also classed as **hardware (external)** vs **software (trap/syscall)**, and **internal (exceptions)**.
- **Vectored interrupts** supply the **ISR address** directly via an **interrupt vector**, avoiding polling.
- **Non-vectored interrupts** jump to a **fixed** ISR address that then **polls** to find the source.
- The **interrupt vector table (IVT)** stores **ISR addresses** indexed by **interrupt number**.
- **Interrupt latency** is the time from **interrupt request** to the **start of the ISR**.
- **Daisy chaining** resolves priority in hardware: the device **closest to the CPU** has the **highest** priority.
- **Software polling** priority checks devices in a **fixed program order**, the **first** polled having highest priority.
- A **priority interrupt controller (PIC, e.g., 8259)** manages **8** interrupt lines and can be **cascaded** to 64.
- **Nested interrupts** allow a **higher-priority** interrupt to preempt a running **lower-priority** ISR.
- The CPU checks for pending interrupts at the **end of each instruction cycle**.

Direct Memory Access (DMA)

- A **DMA controller** needs **3** key registers: **address register**, **count/word-count register**, and **control/status register**.
- DMA setup by the CPU specifies **4** items: **operation (read/write)**, **device address**, **memory start address**, and **word count**.
- **DMA** signals **completion** to the CPU with an **interrupt** after the block transfer.
- **DMA** requires the **HOLD/BUS REQUEST** signal and grants control via **HLDA/BUS GRANT**.
- The **3** DMA transfer modes are **cycle stealing**, **burst (block) mode**, and **transparent/demand** mode.
- **Cycle stealing** transfers **1 word** per bus cycle, **stealing** cycles between CPU memory accesses.
- **Burst mode** transfers the **entire block** while holding the bus, giving **highest throughput** but stalling the CPU.
- **Transparent mode** transfers only when the CPU is **not using** the bus, with **zero** CPU slowdown.
- **DMA** causes **cache coherence** and **bus contention** issues that must be managed.
- **DMA** is faster than interrupt/programmed I/O because it removes **per-word CPU involvement**.

I/O Buses and Transfer

- **PCI** and **PCIe** are **parallel** (PCI) and **serial point-to-point** (PCIe) expansion buses.
- **USB** is a **serial, hot-pluggable** bus organized as a **tiered-star** topology with up to **127** devices.
- **Synchronous** transfer uses a **shared clock**; **asynchronous** transfer uses **handshaking (request/acknowledge)**.
- **Handshaking** uses **2** control signals for **4-way** reliable asynchronous transfer.
- An **I/O processor (IOP/channel)** is a dedicated processor that executes **channel programs** to offload I/O.

Computer Arithmetic (Extended)

Number Representation

- There are **3** signed-integer representations: **sign-magnitude**, **1's complement**, and **2's complement**.
- An **n-bit 2's complement** number ranges from $-2^{(n-1)}$ to $+2^{(n-1)} - 1$, giving **one** extra negative value.
- An **n-bit 1's complement** and **sign-magnitude** both range $-(2^{(n-1)}-1)$ to $+(2^{(n-1)}-1)$ with **two** representations of zero.
- An **n-bit unsigned** number ranges from **0** to $2^n - 1$ and represents 2^n distinct values.
- The **2's complement** of a number is found by **inverting all bits and adding 1**, or by **copying up to the first 1 then inverting the rest**.
- **2's complement** is preferred because it has **one** zero and uses the **same adder** for addition and subtraction.
- In **2's complement**, subtraction $A - B$ is done as $A + (2's\ complement\ of\ B)$.
- The **Most Significant Bit (MSB)** acts as the **sign bit**: **0** for positive and **1** for negative.
- **Sign extension** replicates the **sign bit** to the left when widening a **2's complement** number.

Overflow and Carry

- **Overflow** in 2's complement addition occurs when **two same-sign** operands produce a **different-sign** result.
- **Overflow** can be detected when the **carry into** the MSB differs from the **carry out** of the MSB.
- **Carry** matters for **unsigned** arithmetic, while **overflow** matters for **signed (2's complement)** arithmetic.
- Adding two **n-bit** numbers can need **n+1** bits to store the **unsigned** sum.

Multiplication and Division

- **Booth's algorithm** multiplies **signed 2's complement** numbers by examining **2 bits (Q0 and Q-1)** at a time.
- **Booth's algorithm** performs $A = A - M$ on a **1→0** transition... actually on **10**, $A = A + M$ on **01**, and **no add/sub** on **00/11**.
- **Booth's algorithm** reduces operations for strings of **consecutive 1's**, replacing many adds with **one subtract and one add**.
- **Restoring division** restores the remainder when it goes **negative**, doing **1 subtraction** and possibly **1 addition** per step.
- **Non-restoring division** avoids the restore step, doing **exactly 1** add-or-subtract per bit, making it **faster**.
- An **n-bit** multiplication of two numbers yields a product of up to **2n** bits.
- A **sequential (shift-add) multiplier** needs **n** cycles for **n-bit** operands.
- An **array multiplier** uses a grid of **AND gates and full adders** for **combinational** parallel multiplication.

Floating-Point (IEEE 754)

- **IEEE 754 single precision** uses **32 bits**: **1** sign, **8** exponent, and **23** mantissa bits.
- **IEEE 754 double precision** uses **64 bits**: **1** sign, **11** exponent, and **52** mantissa bits.
- The **exponent bias** is **127** for single precision and **1023** for double precision.
- A **normalized** IEEE 754 number has an **implicit leading 1** (hidden bit) before the mantissa.
- **IEEE 754** represents zero with exponent **all 0s** and mantissa **all 0s**, and there are **two** zeros (**+0, -0**).
- **Infinity** is encoded with exponent **all 1s** and mantissa **all 0s** in **IEEE 754**.
- **NaN (Not a Number)** has exponent **all 1s** and a **non-zero** mantissa.
- **Denormalized** numbers use exponent **all 0s** with **no** implicit leading 1 to represent values near **zero**.
- The value of a normalized IEEE 754 number is $(-1)^S \times 1.M \times 2^{(E - \text{bias})}$.
- **Floating-point addition** requires **3** steps: **align exponents**, **add mantissas**, and **normalize** the result.
- **Fixed-point** has constant precision while **floating-point** trades precision for a **wider** dynamic range.

CBT Trap

- Do not confuse **bias 127** (single) with **1023** (double); using the **wrong bias** is a common error.
- Remember the hidden bit: a **23-bit** stored mantissa gives **24 bits** of effective precision in single precision.

Instruction Set & Performance (Extended)

Performance Metrics

- **CPU execution time** = **Instruction Count x CPI x Clock cycle time**.
- **CPI** stands for **Cycles Per Instruction**, and **IPC** is its reciprocal **Instructions Per Cycle**.
- **MIPS** = **Clock rate / (CPI x 10⁶)**, measuring **millions of instructions per second**.
- **Clock cycle time** = **1 / Clock frequency**, so **1 GHz** equals a **1 ns** cycle.
- **Amdahl's law** bounds speedup: **Speedup = 1 / ((1-f) + f/s)** where **f** is the enhanced fraction.
- The **3** factors in the **CPU performance equation** are **instruction count**, **CPI**, and **clock cycle time**.

RISC vs CISC

- **RISC** uses **fixed-length**, simple instructions executed in **1 cycle**, while **CISC** uses **variable-length** complex instructions.
- **RISC** relies on **many registers** and a **load-store** architecture; **CISC** allows **memory operands** in most instructions.
- **RISC** favors **hardwired** control; **CISC** historically uses **microprogrammed** control.
- A **load-store architecture** permits memory access ONLY through **load** and **store** instructions.

Instruction Formats

- The **4** instruction format types by operand count are **zero**, **one**, **two**, and **three-address**.
- **Zero-address** instructions use a **stack** (operands implicit at **top of stack**).
- **One-address** instructions use an implied **accumulator** register.
- **Stack** expressions are evaluated using **postfix (reverse Polish)** notation.

Addressing Modes (Extended)

- **Immediate** mode stores the operand **within the instruction** itself, needing **zero** memory references for the operand.
- **Direct** mode gives the operand's **effective address** in the instruction, needing **1** memory reference.
- **Indirect** mode stores the **address of the address**, needing **2** memory references.
- **Register** mode keeps the operand **in a register**, the **fastest** access with **no** memory reference.
- **Register indirect** mode holds the operand's **address in a register**.
- **Indexed** mode computes **EA = base + index register**, ideal for **array** access.
- **Base + displacement** mode computes **EA = base register + offset**, used for **record/struct** fields.
- **PC-relative** mode computes **EA = PC + offset**, enabling **position-independent** branches.
- **Auto-increment/decrement** modes adjust the pointer register **after/before** access, ideal for **stacks and arrays**.

Pipelining (Extended)

Pipeline Performance

- A **k-stage** pipeline gives an ideal speedup of **k** over a non-pipelined processor.
- For **n** instructions in a **k-stage** pipeline, total cycles = **k + (n-1)**.
- Pipeline **speedup** = $n \times k / (k + n - 1)$, approaching **k** as **n** -> infinity.
- Pipeline **throughput** = **1 instruction per clock cycle** in the ideal (no-stall) case.
- The **clock period** of a pipeline equals the **slowest stage** delay plus **latch/register** overhead.
- **Pipeline efficiency** = **Speedup / number of stages (k)**.

Pipeline Hazards

- The **3** pipeline hazard types are **structural**, **data**, and **control (branch)** hazards.
- **Structural hazards** arise from **resource conflicts**, solved by **duplicating** hardware or **stalling**.
- **Data hazards** come in **3** kinds: **RAW (read-after-write)**, **WAR**, and **WAW**.
- **RAW** is a **true** dependency; **WAR** is an **anti** dependency; **WAW** is an **output** dependency.
- **Data forwarding (bypassing)** resolves many **RAW** hazards without **stalling**.
- **Control hazards** occur on **branches**, resolved by **branch prediction**, **delayed branch**, or **flushing**.
- **Branch prediction** can be **static** (always taken/not-taken) or **dynamic** (history-based with a **BTB**).
- A **2-bit saturating counter** predictor changes prediction only after **2** consecutive mispredictions.
- The **branch penalty** is the number of **cycles lost/flushed** when a branch is **mispredicted**.

CBT Trap

- Distinguish **latency** (time for **1** instruction) from **throughput** (instructions per unit time); pipelining improves **throughput**, not single-instruction latency.

Memory Hierarchy (Extended)

Cache Mapping Numericals

- **Direct-mapped** cache splits the address into **Tag**, **Line/Index**, and **Block-offset** fields.
- **Fully associative** cache has **no index** field, using only **Tag** and **offset**.
- **Set-associative** cache splits the address into **Tag**, **Set index**, and **offset**.
- Block-offset bits = $\log_2(\text{block size in bytes})$.
- For direct-mapped, index bits = $\log_2(\text{number of cache lines})$.
- For k-way set-associative, set-index bits = $\log_2(\text{number of sets})$, where **sets** = **lines / k**.
- **Tag bits** = **address bits - index bits - offset bits**.
- **Average Memory Access Time (AMAT)** = **Hit time + Miss rate x Miss penalty**.
- **Effective access time** with hit ratio **h** = **h x cache time + (1-h) x memory time**.
- **Hit ratio** = **hits / total accesses**; **miss ratio** = **1 - hit ratio**.

Cache Policies

- The **3** cache miss types ("3 Cs") are **compulsory**, **capacity**, and **conflict** misses.
- **Compulsory** misses are the **first** reference to a block; **conflict** misses come from **mapping** collisions.
- The **2** write-hit policies are **write-through** and **write-back**.
- **Write-through** updates **cache and memory** together; **write-back** updates memory only on **eviction** using a **dirty bit**.
- The **2** write-miss policies are **write-allocate** and **no-write-allocate**.
- Common cache replacement policies are **LRU**, **FIFO**, **Random**, and **LFU**.
- **Spatial locality** favors **larger** block sizes; **temporal locality** favors **caching recently used** data.

Main Memory

- **SRAM** uses **6 transistors** per cell, is **fast**, and needs **no refresh**; **DRAM** uses **1 transistor + 1 capacitor** and needs **periodic refresh**.
- **SRAM** is used for **cache**; **DRAM** is used for **main memory**.
- **Memory interleaving** spreads addresses across **multiple banks** to allow **parallel** access.
- A memory of 2^n words needs **n** address lines.
- **ROM** types include **PROM**, **EPROM (UV-erasable)**, and **EEPROM/Flash (electrically erasable)**.

CBT Trap

- The number of **tag bits** differs across mapping schemes for the **same** cache size; fully associative has the **most** tag bits.

Control Unit (Extended)

- The **2** control unit designs are **hardwired** and **microprogrammed**.
- **Hardwired** control is **fast** but **hard to modify**; **microprogrammed** is **flexible** but **slower**.
- **Microprogrammed** control stores **microinstructions** in **control memory (ROM)**.
- **Horizontal** microcode uses **long**, less-encoded words with **more parallelism**; **vertical** microcode uses **short**, encoded words.
- The **control memory** address is held by the **Control Address Register (CAR)** and data by the **Control Data/Buffer Register (CDR)**.
- A **microinstruction** contains **control signals** plus **next-address** information.

Exam Rapid-Fire Numericals

- A **1 GHz** clock has a cycle time of **1 ns**; a **2 GHz** clock has **0.5 ns**.
- A processor running at **f Hz** executes **f** cycles per second.
- **Speedup** from Amdahl's law is capped at $1/(1-f)$ when **s $\rightarrow$ infinity**.
- A **k-stage** pipeline filling for **n** instructions wastes **k-1** cycles in the **fill** phase.
- **Memory bandwidth** = **bus width (bytes) x bus frequency**.
- A byte-addressable memory with **32-bit** addresses can address **4 GB**.
- **DMA** reduces CPU overhead most for **large block** transfers due to **zero per-word** CPU work.

System Buses & Data Paths (Extended)

- A **system bus** has **3** component buses: the **data bus**, **address bus**, and **control bus**.
- The **address bus** width of **n** lines allows addressing 2^n locations.
- The **data bus** width determines how many **bits** are transferred **per cycle**.
- **Bus arbitration** methods include **daisy chaining**, **polling**, and **independent request** schemes.
- **Synchronous** buses use a **common clock**; **asynchronous** buses use **handshaking** signals.
- **Tri-state buffers** let **multiple devices** share a bus by entering a **high-impedance** state.
- The **3** register-transfer micro-operations are **arithmetic**, **logic**, and **shift** operations.
- A **single-bus** CPU organization needs **more cycles** but is **cheaper**; a **multi-bus** design is **faster**.
- The **instruction cycle** has **4** basic phases: **fetch**, **decode**, **execute**, and **write-back**.
- The **fetch** phase performs **MAR $\leftarrow$ PC**, **MDR $\leftarrow$ M[MAR]**, **IR $\leftarrow$ MDR**, and **PC $\leftarrow$ PC + 1**.

Secondary Storage (Extended)

- **Disk access time** = **seek time** + **rotational latency** + **transfer time**.
- **Average rotational latency** = **half** the time for **one full rotation**.
- A disk rotating at **R** RPM has a full rotation time of **60/R** seconds.
- **Seek time** is the time to move the **read/write head** to the correct **track/cylinder**.
- **Disk capacity** = **surfaces x tracks/surface x sectors/track x bytes/sector**.
- A **cylinder** is the set of **tracks** at the same **radius** across all **platters**.
- **RAID 0** provides **striping** (no redundancy); **RAID 1** provides **mirroring**.
- **RAID 5** uses **distributed parity** and tolerates **1** disk failure; **RAID 6** tolerates **2**.

Parallelism & Misc (Extended)

- **Flynn's taxonomy** defines **4** classes: **SISD**, **SIMD**, **MISD**, and **MIMD**.
- **SIMD** executes **one instruction** on **multiple data** (vector/GPU); **MIMD** runs **many instructions** on **many data** (multicore).
- **Superscalar** processors issue **multiple instructions** per cycle using **multiple pipelines**.
- **VLIW** packs **multiple operations** into **one long instruction**, relying on the **compiler**.
- **Locality of reference** has **2** forms: **temporal** (reuse soon) and **spatial** (nearby addresses).
- **Little-endian** stores the **least-significant byte** at the **lowest** address; **big-endian** stores the **most-significant** byte first.
- A **word** is the natural data unit, commonly **32 or 64 bits** depending on the **architecture**.
- **Memory-mapped I/O** consumes part of the **main memory address space** for **device registers**.